



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
**Proyecto de una Plataforma
Virtual**

Alumno(s):
Manuel David Castro Blanco
Bryant Maximiliano Dzul Perez
Pablo Leonel Salomon Campos
Gerardo Alexander Díaz León

Propósito de este documento

Estas son las reglas de **Clean Code** que el equipo se compromete a seguir durante **todo** el desarrollo de Ajoalnaj. No son reglas copiadas de internet: se eligieron a partir del estado actual de nuestro sistema (Laravel 12 + PHP 8.2 + PostgreSQL, con arquitectura modular por dominios en `app/Domain/*`) y de las decisiones técnicas que ya tomamos en las Fases 1–7. El objetivo es mantener un código **legible, mantenible y consistente** para todo el equipo.

Regla 1 — Nombres descriptivos y en el lenguaje del dominio

La regla. Clases, métodos, variables y rutas se nombran de forma descriptiva y usando el vocabulario real del albergue (español). Un nombre debe decir *qué* es o *qué* hace, sin abreviaturas ambiguas.

- Clases de servicio con verbo + sustantivo: RegistrarIngresoService, AbrirExpedienteService, RegistrarEgresoService.
- Método público de acción del servicio: ejecutar().
- Entidades con el nombre del negocio: PersonaUsuaría, Expediente, Estancia, PeriodoEstancia, Cama.

¿Por qué la implementamos? Nuestro sistema modela un proceso real (ingreso → estancia → seguimiento clínico → egreso). Si el código habla el mismo idioma que el negocio, cualquier integrante entiende qué hace una clase sin leer su implementación.

¿Qué problema evita? La ambigüedad y la “traducción mental” entre el negocio y el código (`$x`, `$data`, `proc()`), que provoca errores y hace lento el mantenimiento.

¿Cómo la aplicamos en el proyecto? Todo servicio de dominio se nombra `<Acción><Entidad>Service` y expone `ejecutar()`. Los modelos usan el nombre del dominio. Las tablas, columnas y roles ya siguen esta convención (`persona_usuario`, `numero_episodio`, roles `trabajo_social` / `enfermeria` / `psicologia`).

Regla 2 — La lógica de negocio vive en Servicios de dominio, no en los controladores

La regla. Los controladores son **delgados**: reciben la petición, delegan el trabajo a un Servicio de dominio (`app/Domain/*/Services`) y devuelven la respuesta. Ninguna regla de negocio se escribe dentro de un controlador.

¿Por qué la implementamos? La lógica del albergue (cómo se calcula un reintegro, cómo se asigna una cama, cómo se cierra una estancia) es el corazón del sistema y debe poder reutilizarse y probarse de forma aislada.

¿Qué problema evita? Los “controladores Dios” que hacen de todo y la duplicación de lógica entre pantallas. También evita que una misma regla quede escrita de dos maneras distintas.

¿Cómo la aplicamos en el proyecto? RegistrarIngresoService::ejecutar() orquesta — dentro de una transacción— la creación de la Estancia, su PeriodoEstancia inicial y la asignación de Cama, calculando el numero_episodio del reingreso. El controlador solo valida (vía FormRequest) y llama al servicio.

Regla 3 — Nada de números ni cadenas “mágicas”: usar constantes con nombre

La regla. Ningún valor de negocio se escribe “a mano” repartido por el código. Se declara **una sola vez** como constante con nombre en el modelo correspondiente.

¿Por qué la implementamos? Reglas como “la permanencia inicial es de 3 meses y el tope es de 6” son decisiones del negocio que pueden cambiar; deben vivir en un único lugar.

¿Qué problema evita? Valores repetidos e inconsistentes (un 3 aquí, un “Usuario” allá) que, al cambiar, se olvidan en algún archivo y generan bugs difíciles de rastrear.

¿Cómo la aplicamos en el proyecto? Constantes reales ya en el código:

```
// app/Domain/Estancias/Models/Estancia.php
public const ESTATUS_USUARIO      = 'Usuario';
public const ESTATUS_RESIDENTE    = 'Residente';
public const MESES_PERIODO_INICIAL = 3;
public const MESES_TOPE_PERMANENCIA = 6;
```

Los servicios usan Estancia::MESES_PERIODO_INICIAL en lugar del número 3.

Regla 4 — Validación y autorización fuera del controlador (FormRequest / Policy)

La regla. Las reglas de “qué datos son válidos” y “quién tiene permiso” viven en un FormRequest (métodos rules() y authorize()) o en una Policy, nunca sueltas dentro del controlador.

¿Por qué la implementamos? Ajoolnaj tiene un RBAC propio con 7 roles y reglas de autorización por área clínica (Psicología no puede escribir el diagnóstico médico, etc.). Esa lógica debe estar centralizada y ser reutilizable.

¿Qué problema evita? Reglas de validación/seguridad dispersas y duplicadas, y controladores inseguros donde es fácil olvidar una comprobación de permisos.

¿Cómo la aplicamos en el proyecto? StoreDiagnosticoRequest define rules() (el área es obligatoria y válida, la descripción no excede 5000 caracteres) y authorize() (el usuario puede editar esa área, con cortocircuito para admin). El controlador recibe la petición ya validada y autorizada.

```
// app/Http/Requests/Clinico/StoreDiagnosticoRequest.php
public function authorize(): bool
{
    $user = $this->user();
    return $user->hasRole('admin')
```

```

    || app(DiagnosticoPolicy::class)->puedeEditarArea($user, (string) $this-
    >input('area'));
}

```

Regla 5 — Comentarios que explican el por qué, y PHPDoc con tipos

La regla. El código explica el *qué*; los comentarios se reservan para el *por qué* (la intención o la regla de negocio). Los servicios llevan PHPDoc con los tipos de sus parámetros y su retorno. No se escriben comentarios obvios.

¿Por qué la implementamos? Hay decisiones de negocio que no son evidentes leyendo el código (p. ej. que un reintegro reutiliza el expediente pero incrementa el episodio).

Documentar la intención evita que alguien “corrija” algo que en realidad es correcto.

¿Qué problema evita? Los comentarios obvios (`// crea la estancia`) que envejecen, se desincronizan del código y terminan mintiendo.

¿Cómo la aplicamos en el proyecto? Cada servicio abre con un docblock que resume su responsabilidad y la regla de negocio, y comentarios puntuales explican decisiones no evidentes:

```

// Reingreso: numero_episodio = max(previos del expediente) + 1.
$ultimoEpisodio = Estancia::where('expediente_id', $expediente->id)-
>max('numero_episodio') ?? 0;

```

Convenciones de estilo complementarias

Además de las 5 reglas principales, el equipo acuerda estos lineamientos de apoyo, alineados con la lista de aspectos sugerida en la actividad:

Aspecto	Convención acordada
Nombres de variables	camelCase para variables/propiedades, en español y descriptivas (\$fechaEgresoPrevista, \$ultimoEpisodio).
Nombres de funciones/métodos	Verbo en infinitivo + sustantivo (ejecutar, puedeEditarArea, asignarCama). Clases en PascalCase.
Organización de carpetas	Modular por dominio: app/Domain/<Dominio>/{Models,Services,Policies}. Nada de “carpetas cajón”.
Longitud de funciones	Funciones cortas y de responsabilidad única; si un método crece, se extrae a otro servicio/método privado.
Uso de comentarios	Solo para el por qué (ver Regla 5). Prohibido comentar lo obvio o dejar código muerto comentado.
Manejo de constantes	Valores de negocio como constantes con nombre en el modelo (ver Regla 3). Nada de “magic numbers/strings”.

Aspecto	Convención acordada
Formato y estilo del código	Estándar PSR-12, aplicado automáticamente con Laravel Pint antes de cada commit (formato uniforme para todo el equipo).

Compromiso del equipo

Estas reglas son de cumplimiento obligatorio y forman parte de nuestra **Definition of Done**: ningún cambio se considera terminado si no las respeta. Su cumplimiento se verifica en la revisión por Pull Request antes de integrar a la rama principal.